СПбГУ · МКН · СП · 1 КУРС

Математические основы алгоритмов

Конспект лекций

Лектор: Косолобов Д. А.

Составители: ....

Санкт-Петербург, 2026

Вторая часть курса

1. Быстрое преобразование Фурье (FFT).

Свёртка и умножение многочленов

Определение 1

Пусть заданы последовательности чисел $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Определим последовательность $c_0, \dots, c_{2n-2}$ по формуле

$$c_k = \sum_{t=0}^{n-1} a_t\, b_{k-t}, \qquad k = 0, \dots, 2n-2,$$

(считаем $a_i = b_i = 0$ при $i \notin [0, n-1]$). Эта операция называется **инволюцией (свертткой)**:

$$c = a * b.$$

Умножение многочленов

Рассмотрим многочлены:

$$A(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1},$$

$$B(x) = b_0 + b_1 x + \dots + b_{n-1} x^{n-1}.$$

Тогда их произведение:

$$C(x) = A(x)B(x) = c_0 + c_1 x + \dots + c_{2n-2} x^{2n-2},$$

где коэффициенты c_k задаются инволюцией.

Сложность

- Наивное умножение: $O(n^2)$.
- Карацуба: $O\big(n^{\log_2 3}\big)$.
- FFT (быстрое преобразование Фурье): $O(n \log n)$.

Идея FFT-умножения

Хотим быстро вычислить свёртку (а значит и произведение многочленов). Для этого:

1. Выбираем число точек N так, чтобы $N \geq 2n - 1$ и обычно $N = 2^t$. Коэффициенты A, B дополняем нулями до длины N.
2. Быстро считаем значения в N точках:

   $$A(\omega_k), \quad B(\omega_k), \qquad k = 0, \dots, N-1,$$

   где ω — примитивный N-й корень из единицы.
3. Перемножаем покомпонентно:

   $$y_k = A(\omega_k)\, B(\omega_k).$$

4. По значениям y_k восстанавливаем коэффициенты C обратным FFT.

Ключевой факт: можно выбрать точки ω_k так, что и «вычисление значений», и «восстановление коэффициентов» делаются за $O(N \log N)$.

Реализация FFT

Далее считаем, что $N = 2^t$. Если исходная длина не степень двойки, дополняем нулями.

Прямое FFT: из коэффициентов в значения

Выбираем

$$\omega = e^{2\pi i/N} = \cos\frac{2\pi}{N} + i\sin\frac{2\pi}{N}, \qquad \omega_k = \omega^k.$$

Нужно вычислить

$$y_k = A(\omega^k) = \sum_{j=0}^{N-1} a_j(\omega^k)^j, \qquad k = 0, \dots, N-1.$$

Разобьём A на чётные и нечётные коэффициенты:

$$A_0(x) = a_0 + a_2 x + \dots + a_{N-2}x^{\frac{N}{2}-1}, \qquad A_1(x) = a_1 + a_3 x + \dots + a_{N-1}x^{\frac{N}{2}-1}.$$

Тогда

$$A(x) = A_0(x^2) + xA_1(x^2).$$

Подставим $x = \omega^k$:

$$A(\omega^k) = A_0(\omega^{2k}) + \omega^k A_1(\omega^{2k}).$$

Обозначим

$$y_k^{(0)} = A_0(\omega^{2k}), \qquad y_k^{(1)} = A_1(\omega^{2k}).$$

Так как значения ω^{2k} пробегают только $N/2$ различных точек, достаточно посчитать $y_k^{(0)}, y_k^{(1)}$ для $k = 0, \dots, \frac{N}{2} - 1$ (рекурсивно), а затем «склеить» ответы:

$$y_k = y_k^{(0)} + \omega^k\, y_k^{(1)}, \qquad y_{k+\frac{N}{2}} = y_k^{(0)} - \omega^k\, y_k^{(1)}.$$

(Здесь использовано $\omega^{k+N/2} = -\omega^k$.)

Оценка времени прямого FFT

На каждом уровне рекурсии решаем 2 подзадачи размера $N/2$ и делаем склейку за $O(N)$:

$$T(N) = 2T\left(\frac{N}{2}\right) + O(N) = O(N \log N).$$

Обратное FFT: из значений в коэффициенты

Прямое преобразование можно записать матрично:

$$\begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^1 & (\omega^1)^2 & \cdots & (\omega^1)^{N-1} \\ 1 & \omega^2 & (\omega^2)^2 & \cdots & (\omega^2)^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & (\omega^{N-1})^2 & \cdots & (\omega^{N-1})^{N-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix}.$$

Это матрица Вандермонда F_ω.

Утверждение 1

Выполнено
$$F_{\omega^{-1}} F_\omega = N \cdot \mathbf{I}.$$

Доказательство. Элемент в позиции (i, j) равен
$$\sum_{k=0}^{N-1} (\omega^i)^k (\omega^{-j})^k = \sum_{k=0}^{N-1} (\omega^{i-j})^k.$$

Если $i \neq j$, то $q := \omega^{i-j} \neq 1$, и по формуле суммы геометрической прогрессии
$$\sum_{k=0}^{N-1} q^k = \frac{1 - q^N}{1 - q}$$
получаем
$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \frac{1 - (\omega^{i-j})^N}{1 - \omega^{i-j}} = \frac{1 - (\omega^N)^{i-j}}{1 - \omega^{i-j}} = \frac{1 - 1}{1 - \omega^{i-j}} = 0.$$
Если $i = j$, то
$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \sum_{k=0}^{N-1} 1 = N.$$

■

Следовательно, обратное преобразование — это прямое FFT с корнем ω^{-1}, после чего нужно поделить все коэффициенты на N:
$$a = \frac{1}{N} F_{\omega^{-1}} y.$$

Сложность умножения многочленов через FFT

Нужно:

- одно прямое FFT для A,
- одно прямое FFT для B,
- покомпонентное умножение ($O(N)$),
- одно обратное FFT.

Итого $O(N \log N)$, где N — ближайшая степень двойки, не меньшая $2n - 1$.

При этом $2n - 1 \leq N < 2(2n - 1) < 4n$, то есть $N = \Theta(n)$, поэтому итоговая сложность равна $O(n \log n)$.

Обобщение: NTT (Number-Theoretic Transform)

Вместо $\mathbb{C}$ можно работать в кольце/поле R, если:

- в R существует элемент ω порядка N (примитивный N-й корень из единицы);
- число N обратимо в R (нужно для деления на N в обратном преобразовании).

Пример (поле $\mathbb{Z}_p$)

Пусть $R = \mathbb{Z}_p$, где p — простое, и

$$N = 2^k, \qquad p = c \cdot 2^k + 1.$$

Тогда $p - 1$ кратно N, и существует первообразный корень g по модулю p (порождает мультипликативную группу порядка $p - 1$). Можно взять

$$\omega \equiv g^c \pmod p,$$

тогда ω имеет порядок N, и NTT работает полностью по модулю p.

Приложение: смысл коэффициентов DFT

Пусть дан сигнал (последовательность)

$$a_0, a_1, \ldots, a_{N-1}.$$

Определим дискретное преобразование Фурье:

$$y_k = \sum_{t=0}^{N-1} a_t \, \omega^{kt}, \qquad k = 0, \ldots, N-1, \quad \text{где} \;\; \omega = e^{2\pi i/N}.$$

Так как

$$\omega^{kt} = \cos\left(\frac{2\pi k}{N} t\right) + i \sin\left(\frac{2\pi k}{N} t\right),$$

то

$$\Re(y_k) = \sum_{t=0}^{N-1} a_t \cos\left(\frac{2\pi k}{N} t\right), \qquad \Im(y_k) = \sum_{t=0}^{N-1} a_t \sin\left(\frac{2\pi k}{N} t\right).$$

Эти суммы — скалярные произведения сигнала с косинусом и синусом частоты k. Если в сигнале сильно выражена гармоника частоты k, то вклады «складываются», и модуль $|y_k|$ получается большим; если такой частоты нет — вклады в основном взаимно компенсируются и $|y_k|$ близок к нулю.

2. Префиксные суммы (префиксное накопление, операция `scan`). Сумматор Ладнера–Фишера.

Задача (prefix sums). Построить вычислительную схему, которая принимает на вход n чисел $x_0, \ldots, x_{n-1}$ и (используя только бинарную операцию “+”) выдаёт числа $y_0, \ldots, y_{n-1}$, где

$$y_i = \sum_{j=0}^{i} x_j, \qquad i = 0, \ldots, n-1.$$

Схема Ладнера–Фишера

Пусть n чётное.

Сначала попарно складываем входы:

$$s_i = x_{2i} + x_{2i+1}, \qquad i = 0, \ldots, \frac{n}{2} - 1.$$

Рекурсивно считаем префиксные суммы для s:

$$p_i = \sum_{j=0}^{i} s_j.$$

Тогда выходы восстанавливаются так:

$$y_{2i+1} = p_i, \qquad y_{2i} = \begin{cases} x_0, & i = 0, \\ p_{i-1} + x_{2i}, & i \geq 1. \end{cases}$$

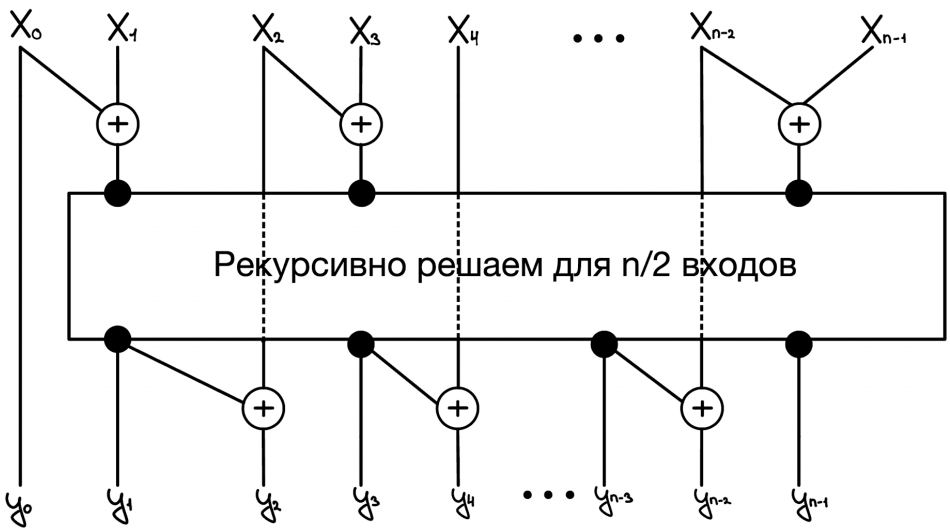

Рис. 1: Схема Ладнера–Фишера для префиксных сумм

Оценки (глубина и размер).

$T(n) = 2+T\left(\frac{n}{2}\right)$, $T(1) = 0 \Rightarrow T(n) = 2\log n$ $(n = 2^m)$, $\quad S(n) = (n-1)+S\left(\frac{n}{2}\right) = 2n-\log n-2.$

Идея хорошо ложится на GPU (много независимых операций на каждом уровне).

Замечание. Операция `scan` (префиксное накопление) определена для любой **ассоциативной** бинарной операции $\odot$ на множестве S (то есть для **полугруппы** $(S, \odot)$):

$$y_i = x_0 \odot x_1 \odot \cdots \odot x_i.$$

Примеры операций $\odot$:

$$(\mathbb{R}, +), \quad (\mathbb{R}, \min), \quad (\mathbb{R}, \max), \quad (\mathbb{R}^{k\times k}, \cdot) \text{ (умножение матриц).}$$

Псевдокод `scan`

Algorithm 1 `scan` (in-place), inclusive, по доске

Require: массив $x[0..n-1]$, где $n = 2^m$

```
                                  ▷ Upsweep: накапливаем суммы на концах блоков

for d = 0 to log_2 n − 1 do   ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
    for k = 0 to n − 1 step 2^{d+1} do
        x[k + 2^{d+1} − 1] = x[k + 2^{d+1} − 1] + x[k + 2^d − 1]
                  ▷ Downsweep: добавляем префикс предыдущего блока к левому подблоку
for d = log_2 n − 1 downto 0 do   ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
    for k = 0 to n − 1 step 2^{d+1} do
        if k > 0 then
            x[k + 2^d − 1] = x[k + 2^d − 1] + x[k − 1]
```

Пример 1: Сведения к `scan`: рекурсия вида $c_{k+1} = a_k + b_k c_k$

Даны числа $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Нужно найти $c_0, \dots, c_n$ такие, что

$$c_0 = 0, \qquad c_{k+1} = a_k + b_k\, c_k.$$

Определим строковый вектор

$$C_k = (1,\ c_k).$$

Тогда

$$C_{k+1} = C_k\, X_k, \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Отсюда

$$C_1 = C_0 X_0, \quad C_2 = C_0 X_0 X_1, \quad C_3 = C_0 X_0 X_1 X_2, \ \dots$$

То есть достаточно посчитать префиксные произведения матриц $X_0, X_1, \dots, X_{n-1}$ (умножение матриц ассоциативно), и это делается через `scan` за $O(\log n)$ времени в параллельной модели.

Замечание. В примере с матрицами внутри умножения матриц одновременно используются две операции над элементами:

$$(A \cdot B)_{ij} = \bigoplus_k (A_{ik} \odot B_{kj}).$$

Поэтому обобщение работает не для одной операции, а для **полукольца** $(S, \oplus, \odot)$: требуется, чтобы

- $\oplus$ ассоциативна (обычно коммутативна) и имеет нейтральный элемент 0;
- $\odot$ ассоциативна и имеет нейтральный элемент 1;
- $\odot$ дистрибутивна относительно $\oplus$: $a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c)$ и $(a \oplus b) \odot c = (a \odot c) \oplus (b \odot c)$;
- 0 поглощает умножение: $0 \odot a = a \odot 0 = 0$.

Важно: **порядок операций существенен** — в роли "сложения" стоит $\oplus$ (левая операция), а в роли "умножения" стоит $\odot$ (правая операция).

Примеры полукольца $(S, \oplus, \odot)$:

$$(\{0,1\},\ \vee,\ \wedge), \qquad (\{0,1\},\ \oplus,\ \wedge), \qquad (\mathbb{R} \cup \{+\infty\},\ \min,\ +).$$

Сумматор Ладнера–Фишера и переносы

Определение 2

Сумматор — это вычислительная схема над булевыми операциями $(\vee, \wedge, \oplus, \neg, \dots)$, которая принимает на вход биты

$$x_0, \dots, x_{n-1}, \qquad y_0, \dots, y_{n-1}$$

и выдаёт биты

$$z_0, \dots, z_n$$

такие, что $z = x + y$, где

$$x = \sum_{k=0}^{n-1} x_k 2^k, \qquad y = \sum_{k=0}^{n-1} y_k 2^k, \qquad z = \sum_{k=0}^{n} z_k 2^k.$$

Биты переноса и сведение к `scan`

Пусть c_k — перенос *в* разряд k (по определению $c_0 = 0$). Тогда для $k = 0, \dots, n-1$ выполняется формула переноса:

$$c_{k+1} = (x_k \wedge y_k)\ \vee\ (x_k \wedge c_k)\ \vee\ (y_k \wedge c_k).$$

Её удобно переписать в виде

$$c_{k+1} = a_k\ \vee\ (b_k \wedge c_k), \qquad a_k := x_k \wedge y_k, \quad b_k := x_k \vee y_k.$$

Если все переносы $c_0, \dots, c_n$ известны, то ответ восстанавливается по формулам

$$z_k = x_k \oplus y_k \oplus c_k \quad (k = 0, \dots, n-1), \qquad z_n = c_n.$$

Ключевая идея: рекуррента

$$c_{k+1} = a_k\ \vee\ (b_k \wedge c_k)$$

имеет вид “аффинного” преобразования, которое можно упаковать в матрицу. Положим

$$C_k = (1,\ c_k), \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Тогда

$$C_{k+1} = C_k X_k,$$

где операции в матрице понимаются над булевым полукольцом $(\{0,1\},\ \vee,\ \wedge)$. Следовательно, нужно посчитать префиксные произведения

$$C_k = C_0 X_0 X_1 \cdots X_{k-1},$$

что делается через `scan` за $O(\log n)$ времени (в параллельной модели).

Модель PRAM

Определение 3

PRAM (Parallel Random Access Machine) — идеализированная модель параллельных вычислений:

- есть **общая глобальная память**;
- есть (теоретически) **неограниченное число процессоров**, каждый процессор — RAM с собственным номером $p = 1, 2, \ldots$;
- процессоры работают **синхронно**: за единицу времени выполняют одну инструкцию.

Разрешение конфликтов доступа к памяти

Для чтения/записи различают режимы **concurrent/exclusive**:

- **concurrent read** — несколько процессоров могут одновременно читать одну ячейку;
- **concurrent write** — несколько процессоров могут одновременно писать в одну ячейку; в модели CRCW нужно отдельно задавать правило разрешения конфликта (например, *arbitrary*: остаётся одно из записанных значений);
- **exclusive** — если несколько процессоров обращаются к одной ячейке "одновременно", это запрещено (поведение не определено/"мусор").

Стандартные варианты PRAM:

$$\text{CRCW PRAM}, \quad \text{CREW PRAM}, \quad \text{ERCW PRAM}, \quad \text{EREW PRAM}.$$

Класс NC

NC — класс задач, которые решаются на PRAM за время $(\log n)^{O(1)}$ при использовании *полиномиального* числа процессоров $n^{O(1)}$, где n – размер входа.

Как в PRAM посчитать `scan` на p процессорах

Пусть дан массив $A[0..n-1]$ и p процессоров. Разобьём массив на блоки длины $m = n/p$ (для простоты считаем $p \mid n$).

1. Каждый процессор i считает **локальный scan** на своём блоке $A[im..(i+1)m-1]$ последовательно за время $O(m) = O(n/p)$. Обозначим сумму блока как T_i (последний элемент локального scan).
2. Считаем **scan по массиву блок-сумм** $T_0, T_1, \ldots, T_{p-1}$ за время $O(\log p)$. Пусть
$$P_i = \sum_{j=0}^{i-1} T_j$$
— сумма всех предыдущих блоков (смещение для блока i).
3. Каждый процессор прибавляет смещение P_i ко всем элементам своего блока (снова $O(n/p)$ времени).

Итого время:

$$O\left(\frac{n}{p} + \log p\right).$$

В частности, при $p = \Theta\left(\frac{n}{\log n}\right)$ получаем $O(\log n)$ времени.

Пример 2: Сортировка подсчётом (Counting sort) в PRAM

Дан массив $a[0..n-1]$, причём $a[k] \in \{0, \dots, U\}$, где $U = O(1)$. Есть p процессоров, нумеруем их $i = 0, 1, \dots, p-1$. Каждый процессор обрабатывает блок длины $m = \frac{n}{p}$.

- Процессор i строит таблицу частот $c_i[0..U]$, где

$$c_i(x) = \#\{\, k \in [im, (i+1)m - 1] \mid a[k] = x \,\}, \qquad m = \frac{n}{p}.$$

  Это занимает $O(\frac{n}{p})$ времени.
- Далее соберём все частоты в один массив длины $(U+1)p$ в порядке

$$C[xp + i] = c_i(x), \qquad x = 0, \dots, U, \quad i = 0, \dots, p-1,$$

  то есть

$$C = \big(c_0(0), c_1(0), \dots, c_{p-1}(0),\ c_0(1), \dots, c_{p-1}(1),\ \dots,\ c_{p-1}(U)\big).$$

  Посчитаем `scan` (префиксные суммы) по C и получим массив P.
- После этого процессор i знает, сколько элементов значения x находится у процессоров с номерами $\le i$ (это $P[xp + i]$), а значит знает и **диапазон позиций** в выходном массиве для своих элементов значения x:

$$\text{элементы } x \text{ из блока } i \text{ кладём в } \big[\, P[xp+i] - c_i(x),\ P[xp+i) \,\big)$$

  И просто выписываем элементы в эти позиции.

Время. Локальный подсчёт частот занимает $O(n/p)$. `scan` по массиву длины $(U+1)p$ занимает $O(\log((U+1)p)) = O(\log p)$, так как $U = O(1)$. Итого время:

$$O\left(\frac{n}{p} + \log p\right).$$